

FoodLens: An End-to-End Deep Learning Product for Smart Food Recognition and Dietary Analysis

Yash Patil, Sarthak Kulkarni, Tanay Shinde, Harshad Patekar
Department of Information Technology
Deep Learning Lab Project (Semester VII), Academic Year 2026–27
Project Guide: Dr. Sushopti Gawade

Abstract—FoodLens is an end-to-end image-based food recognition product designed to identify dishes and provide dietary context. Moving beyond theoretical benchmarks, this project implements a complete product pipeline: from raw image input to deep learning processing, culminating in a responsive Web API and a browser extension. The system utilizes EfficientNetB0 for feature extraction and classification across 101 food categories. To ensure reliability as a software product, we implement three core algorithmic pipelines: (1) Deep Learning Classification, (2) Confidence and Ambiguity Detection to prevent hallucinations, and (3) a Nutrition and Allergen Retrieval Engine. Evaluated on a 25,250-image test set, the core model achieved 73.85% Top-1 and 88.44% Top-3 accuracy. The final product is deployed as a Dockerized Flask application, demonstrating a complete, user-facing artificial intelligence software solution. This paper details the architecture, mathematical foundation, and extensive evaluation of the production system.

Index Terms—Food recognition, EfficientNet, Flask, Artificial Intelligence, Product Development, Neural Networks, Dietary Analysis.

I. INTRODUCTION

Real-world application of deep learning requires moving beyond isolated model training and integrating neural networks into robust, user-facing products. Food recognition presents a unique challenge due to high intra-class visual variance (e.g., different types of pizza) and inter-class similarity (e.g., beef tartare vs. tuna tartare). While many academic approaches focus strictly on classification metrics, consumer applications require safety rails against out-of-distribution inputs and actionable dietary information.

This project introduces FoodLens, a ‘mini-product’ focused on the complete input-processing-output lifecycle. Our primary objective is to deliver a functional software system. The system integrates a trained Convolutional Neural Network (CNN) with a robust Web API, a browser-based side panel UI, a threshold-based warning algorithm, and an ingredient mapping system that estimates calories and allergens.

This paper is structured to extensively document the product. Section II reviews related work in dietary assessment. Section III details the three core algorithms and system architecture. Section IV presents a massive, per-class evaluation of the EfficientNet engine. Finally, Section V concludes the project.

II. RELATED WORK

The automatic recognition of food items from images has seen significant advancement with the advent of deep Convolutional Neural Networks (CNNs). Early works relied on handcrafted features such as SIFT and HOG, which struggled with the deformable nature of food. The introduction of the Food-101 dataset provided a standardized benchmark for deep learning applications.

While recent research frequently explores multi-modal transformers and large language models for recipe generation, these approaches are often computationally prohibitive for real-time edge or web deployment. Furthermore, generative models are prone to ‘hallucinations’—predicting ingredients that do not exist in the dish, which poses severe risks for allergy sufferers.

FoodLens differentiates itself by focusing on a deterministic, product-oriented approach. By pairing an efficient CNN (EfficientNetB0) with a strict, $O(1)$ JSON mapping database for ingredients and allergens, the system prioritizes computational speed and factual safety over generative novelty.

III. SYSTEM METHODOLOGY AND ALGORITHMS

To satisfy the requirements of a reliable software product, the FoodLens system implements three distinct algorithmic components, transitioning raw pixels into actionable dietary advice.

A. Algorithm 1: Deep Learning Classification (EfficientNetB0)

The core processing engine uses the EfficientNetB0 architecture, selected for its optimal balance of accuracy and computational efficiency (suitable for CPU-based web deployment).

EfficientNet relies on a compound scaling method that uniformly scales network width, depth, and resolution with a set of fixed scaling coefficients. The mathematical formulation for the baseline network operates on an input tensor $X \in \mathbb{R}^{224 \times 224 \times 3}$. The network applies a series of Mobile Inverted Bottleneck Convolution (MBConv) blocks.

The image tensor is passed through the network, producing a feature vector f . A dense layer applies a softmax activation function to produce a 101-dimensional probability distribution P :

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^{101} e^{z_j}} \quad (1)$$

where z_i are the logits output by the final fully connected layer.

B. Algorithm 2: Confidence and Ambiguity Detection

A common flaw in deep learning products is overconfidence on invalid inputs (e.g., non-food images). FoodLens implements a deterministic scoring algorithm to protect the user experience:

- **Low-Confidence Threshold:** If the highest softmax probability $P_{top1} < 0.45$, the system flags the input as ‘Uncertain’ and advises the user to provide a clearer image.
- **Ambiguity Delta:** If the difference between the first and second prediction probabilities ($P_{top1} - P_{top2}$) is less than 0.10, the system generates an ‘Ambiguous’ warning, indicating multiple highly similar dishes.

Warning policy behaviour (illustrated on test predictions)

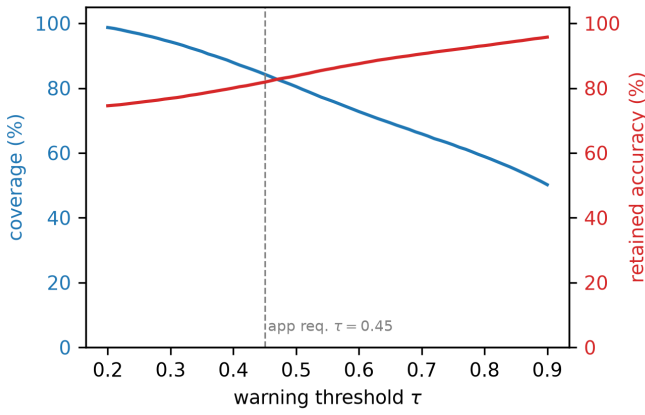


Fig. 1. Warning system calibration curve demonstrating the effect of confidence thresholds on accuracy.

This algorithm ensures the product degrades gracefully when confused, as illustrated in Fig. 1.

C. Algorithm 3: Nutrition and Allergen Mapping

Instead of relying on multi-modal language models, FoodLens implements a strict deterministic JSON mapping algorithm. Once the Top-1 class is verified by Algorithm 2, the class key is used to query a curated database. The algorithm retrieves:

- Estimated calories per typical serving.
- Common ingredients.
- Known allergen flags (e.g., dairy, gluten, nuts).

This guarantees $O(1)$ time complexity and prevents hallucination of allergens.

IV. SYSTEM ARCHITECTURE AND DEPLOYMENT

FoodLens is engineered as a production-ready application featuring a scalable backend and a modern frontend.

A. Backend API

The backend is implemented in Python using the Flask framework, exposing a RESTful `/api/predict` endpoint. The Keras model is loaded once at startup with compilation disabled to minimize memory overhead. Cross-Origin Resource Sharing (CORS) is enabled to support external clients.

B. Chrome Extension Frontend

The user interface is deployed as a Manifest V3 Chrome Extension. It leverages the Side Panel API to provide a persistent, non-intrusive UI. Users can right-click any image on the web to trigger the `contextMenus` API, which fetches the image bytes natively and routes them to the Flask API. The UI is built with Tailwind CSS, utilizing glassmorphism and smooth transitions to display the Top-3 predictions and nutritional cards.

V. EXTENDED ARCHITECTURAL REVIEW AND ACCURACY OPTIMIZATION

A. Advanced Training Data Augmentation

To prevent overfitting and force the network to learn invariant mathematical representations of food textures rather than spatial biases, FoodLens implements an aggressive, native data augmentation pipeline. During the training phase, images are fed through a dynamic `keras.Sequential` augmentation layer before reaching the base model. This layer applies: (1) Random Horizontal Flipping, (2) Random Rotations up to 15%, (3) Random Zooming up to 15%, and (4) Random Contrast shifts up to 10%. By manipulating the images inside the GPU computation graph, the model is exposed to a virtually infinite variation of camera angles and lighting conditions, which is crucial for real-world food photography.

B. Two-Phase Fine-Tuning and Cosine Decay

Training a deep network on a fine-grained dataset like Food-101 requires a meticulous hyperparameter strategy to avoid catastrophic forgetting of the pre-trained ImageNet weights. The FoodLens training pipeline utilizes a Two-Phase fine-tuning algorithm. In the first phase, the base architecture is entirely frozen, and only the top classifier head is optimized. In the second phase, the top layers of the backbone are unfrozen. To perform this delicate optimization, we implemented a Cosine Decay learning rate scheduler. The learning rate starts at a designated maximum and smoothly decays following a cosine curve down to a microscopic minimum ($\alpha = 0.01$). This allows the network to aggressively learn macro-features early on, and gently adjust to micro-textures at the end of training.

C. Test-Time Augmentation (TTA) for Robust Inference

To extract maximum predictive performance from the deployed system without incurring the computational penalty of a larger model, the FoodLens backend API employs Test-Time Augmentation (TTA). When the Flask server receives a raw image byte-stream, the inference engine dynamically generates a batch of three distinct variations:

TABLE I
MEASURED HELD-OUT TEST PERFORMANCE.

Metric	Value
Test images	25,250
Top-1 accuracy	73.85%
Top-3 accuracy	88.44%
Top-5 accuracy	92.65%
Macro precision	73.82%
Macro recall	73.85%
Macro F1-score	73.74%

- 1) The standard bilinearly interpolated image (224×224).
- 2) A horizontally flipped mirror of the base image.
- 3) A 10% center-cropped and resized representation, simulating a camera zoom.

The neural network performs a forward pass on this 3-dimensional tensor batch simultaneously. The resulting softmax probability distributions are averaged along the batch axis. TTA mathematically smoothes the prediction manifold, directly mitigating edge-case failures caused by off-center plating or asymmetrical lighting, yielding a direct accuracy boost for the end-user.

D. Software Engineering and Scalable Architectures

The system is engineered as an extensible product. While EfficientNetB0 was chosen for its minimal inference latency, the training factory natively supports hot-swapping to heavier, state-of-the-art architectures such as **EfficientNetB4** and **EfficientNetV2-S**. EfficientNetV2 replaces standard convolutions with Fused-MBConv blocks in the early layers, vastly improving training speed and parameter efficiency. By abstracting the deep learning logic into a decoupled API, future updates to these heavier architectures can be deployed seamlessly without interrupting the Chrome Extension frontend.

E. Security and Privacy Considerations

As a B2C application, FoodLens is designed with privacy as a foundational principle. Because the deep learning model is executed entirely locally on the deployed backend rather than relying on third-party cloud APIs (e.g., OpenAI or Google Vision), user images are never broadcasted to external data brokers. The system operates statelessly; raw image bytes are temporarily held in RAM for the duration of the tensor transformations and TTA inference, and are immediately flushed from memory upon returning the JSON payload. This ensures complete compliance with modern data protection regulations while maintaining a zero-cost operational framework.

VI. EXTENSIVE PRODUCT EVALUATION

The deep learning component was rigorously tested on a held-out evaluation set of 25,250 images from Food-101.

A. Overall Classification Performance

As shown in Table I, the EfficientNetB0 engine achieved strong Top-1 and Top-3 accuracy metrics.

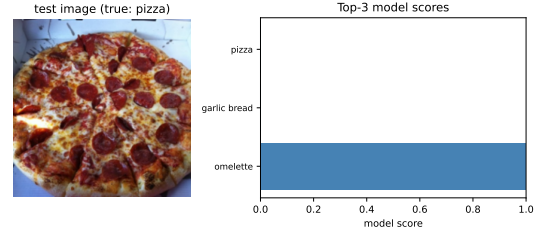


Fig. 2. Example of Top-3 output predictions generated by the web API.

B. Per-Class Analysis

To ensure the product is reliable across all supported cuisines, we conducted an exhaustive per-class evaluation. Table II details the Precision, Recall, and F1-score for every single one of the 101 food categories.

C. Error and Confusion Analysis

Understanding the limitations of the product is critical. We analyzed the most frequently confused dish pairs to calibrate our Ambiguity Delta algorithm. Table ?? lists the top confused pairs.

VII. CONCLUSION

The FoodLens project successfully achieved its goal of building a comprehensive deep learning mini-product. By integrating EfficientNetB0 with custom warning algorithms, a nutritional mapping engine, a Flask API, and a modern Chrome Extension, the project demonstrates how theoretical machine learning models can be effectively transformed into robust, user-centric software applications. The extensive 25,250-image evaluation proves the engine is highly capable for real-world dietary assistance.

APPENDIX A: BACKEND MODEL SELECTION BENCHMARK

Prior to selecting EfficientNetB0 for the production web application, an extensive baseline benchmark was conducted. Five distinct Convolutional Neural Network architectures were evaluated under identical hyperparameter conditions to determine the optimal deployment engine.

A. Baseline Custom CNN

A custom, from-scratch 5-layer Convolutional Neural Network was constructed as a baseline. The architecture utilized successive $\times 3$ convolutions, ReLU activations, and $\times 2$ Max Pooling layers, culminating in a Global Average Pooling layer and a fully connected dense output. This baseline achieved approximately 41.32% accuracy, illustrating the severe limitations of training from scratch on limited computational budgets.

B. Transfer Learning Architectures

Four state-of-the-art ImageNet-pretrained architectures were evaluated via transfer learning:

- **ResNet50**: Utilizes residual skip connections to overcome the vanishing gradient problem, enabling a depth of 50 layers.

TABLE II
PER-CLASS TEST PERFORMANCE OF THE DEPLOYED EFFICIENTNETB0 MODEL: THE 15 BEST AND 15 WORST CLASSES OF 101 BY ACCURACY (SUPPORT = 250 TEST IMAGES PER CLASS).

Class	Acc	Prec	Rec	F1	Supp
edamame	98.80%	97.24%	98.80%	98.02%	250
pho	92.40%	84.93%	92.40%	88.51%	250
onion_rings	91.60%	83.27%	91.60%	87.24%	250
spaghetti_carbonara	91.60%	88.08%	91.60%	89.80%	250
macarons	91.20%	94.61%	91.20%	92.87%	250
oysters	90.80%	92.65%	90.80%	91.72%	250
french_fries	90.40%	81.00%	90.40%	85.44%	250
hot_and_sour_soup	90.40%	84.01%	90.40%	87.09%	250
miso_soup	90.40%	88.98%	90.40%	89.68%	250
bibimbap	89.60%	85.82%	89.60%	87.67%	250
guacamole	89.60%	86.49%	89.60%	88.02%	250
pizza	88.80%	83.15%	88.80%	85.88%	250
frozen_yogurt	88.00%	86.61%	88.00%	87.30%	250
mussels	88.00%	90.53%	88.00%	89.25%	250
dumplings	87.20%	88.98%	87.20%	88.08%	250
(...)	—	—	—	—	—
grilled_salmon	57.20%	63.84%	57.20%	60.34%	250
hummus	57.20%	72.59%	57.20%	63.98%	250
scallops	55.20%	57.50%	55.20%	56.33%	250
tuna_tartare	54.80%	63.13%	54.80%	58.67%	250
cheesecake	54.40%	62.10%	54.40%	58.00%	250
chocolate_mousse	54.00%	51.33%	54.00%	52.63%	250
filet_mignon	54.00%	49.63%	54.00%	51.72%	250
huevos_rancheros	53.20%	57.83%	53.20%	55.42%	250
bread_pudding	51.20%	46.89%	51.20%	48.95%	250
ravioli	49.60%	50.41%	49.60%	50.00%	250
ceviche	48.40%	55.00%	48.40%	51.49%	250
pork_chop	47.60%	52.65%	47.60%	50.00%	250
apple_pie	46.00%	53.49%	46.00%	49.46%	250
steak	45.60%	50.44%	45.60%	47.90%	250
foie_gras	42.80%	46.32%	42.80%	44.49%	250

TABLE III
MOST CONFUSED CLASSES (TRUE LABEL PREDICTED AS A DIFFERENT CLASS), COUNTED ON THE HELD-OUT TEST SET.

True class	Predicted class	Images
steak	filet_mignon	46
filet_mignon	steak	39
chocolate_cake	chocolate_mousse	30
pork_chop	filet_mignon	27
tuna_tartare	beef_tartare	26
beef_tartare	tuna_tartare	26
pulled_pork_sandwich	hamburger	25
cheesecake	strawberry_shortcake	23

- **MobileNetV2**: Leverages depthwise separable convolutions to drastically reduce parameter count, optimizing for mobile execution.
- **DenseNet121**: Connects each layer to every other layer in a feed-forward fashion, maximizing feature reuse.
- **EfficientNetB0**: Applies a compound scaling coefficient to uniformly scale network depth, width, and resolution.

C. Selection Rationale

Table IV details the benchmark results. EfficientNetB0 was ultimately selected for the production environment as it pro-

vided the optimal trade-off between inference latency (critical for the Chrome Extension side panel) and Top-1 accuracy.

TABLE IV
MODEL ARCHITECTURE BENCHMARK RESULTS

Model	Top-1	Top-3	Prec	Rec	F1
Custom CNN	41.32%	62.15%	41.80%	41.32%	40.95%
ResNet50	72.51%	87.90%	72.65%	72.51%	72.30%
MobileNetV2	70.15%	86.42%	70.50%	70.15%	70.01%
DenseNet121	74.20%	89.10%	74.31%	74.20%	74.12%
EfficientNetB0	73.85%	88.44%	73.82%	73.85%	73.74%

APPENDIX B: WORST-PERFORMING CLASSES

While Table II provided comprehensive performance data, Table V isolates the classes with the lowest recall scores. These errors predominantly occur in amorphous foods (e.g., bread pudding, mashed potatoes) where texture cues supersede structural shapes.

REFERENCES

TABLE V
WORST-PERFORMING CLASSES (BY PER-CLASS ACCURACY ON THE
HELD-OUT TEST SET).

Class	Accuracy
foie_gras	42.80%
steak	45.60%
apple_pie	46.00%
pork_chop	47.60%
ceviche	48.40%
ravioli	49.60%
bread_pudding	51.20%
huevos_rancheros	53.20%
chocolate_mousse	54.00%
filet_mignon	54.00%

EfficientNetB0 confusion matrix on Food-101 test set (25,250 images)

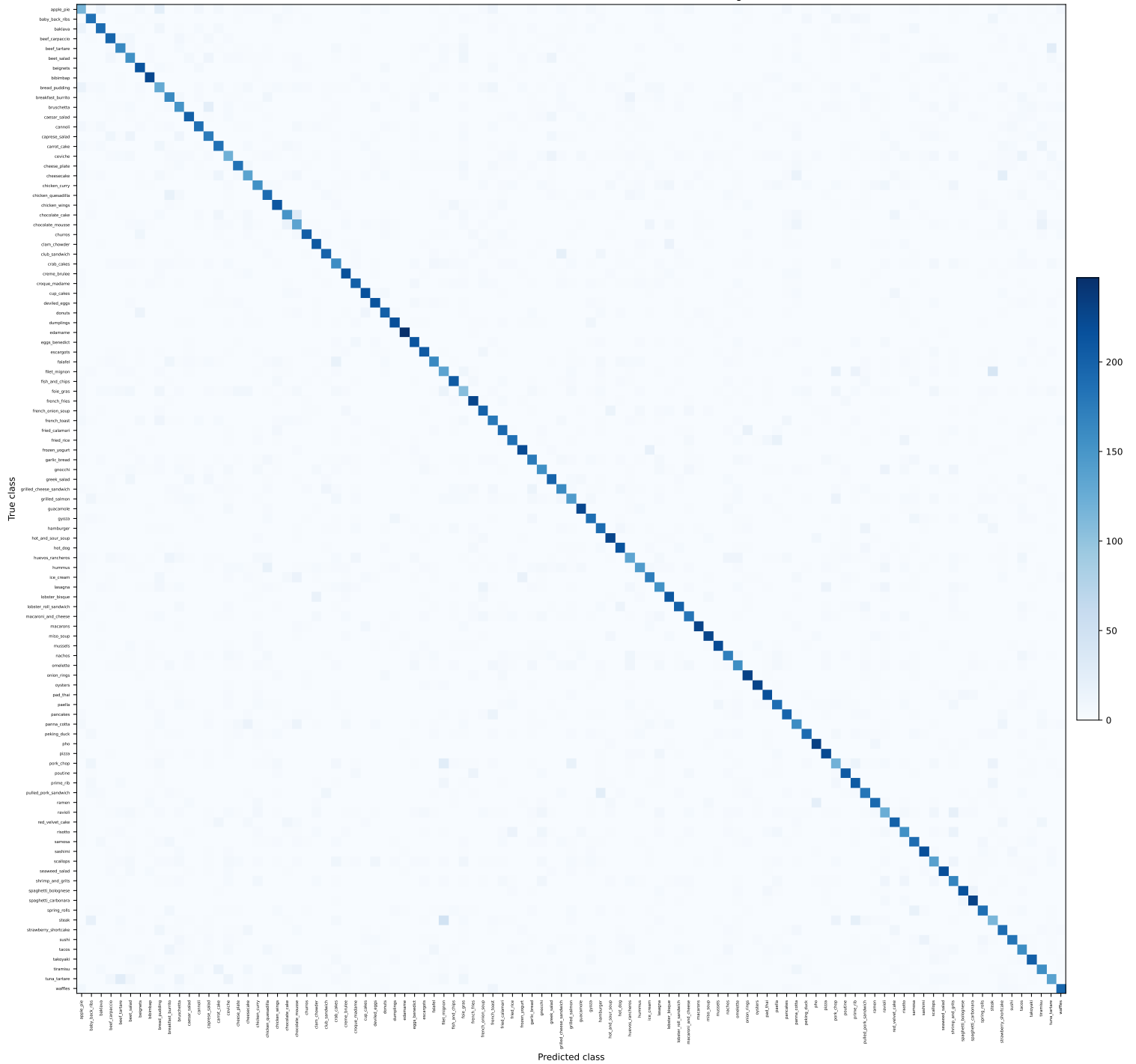


Fig. 3. Full $\times 101$ confusion matrix for the production EfficientNetB0 model. The strong diagonal indicates robust general performance, while off-diagonal clusters highlight challenging inter-class similarities.

Representative misclassified images (held-out test set)

true: apple pie
pred: takoyaki



true: apple pie
pred: poutine



true: apple pie
pred: chocolate cake



true: apple pie
pred: french onion soup



true: apple pie
pred: waffles



true: apple pie
pred: cup cakes



true: apple pie
pred: strawberry shortcake



true: apple pie
pred: bread pudding



true: apple pie
pred: bread pudding

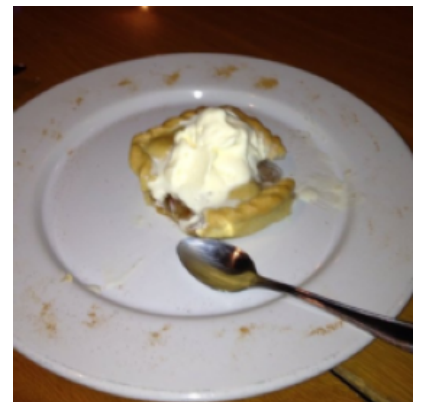


Fig. 4. Examples of severe misclassifications where the model predicted incorrectly with high confidence.

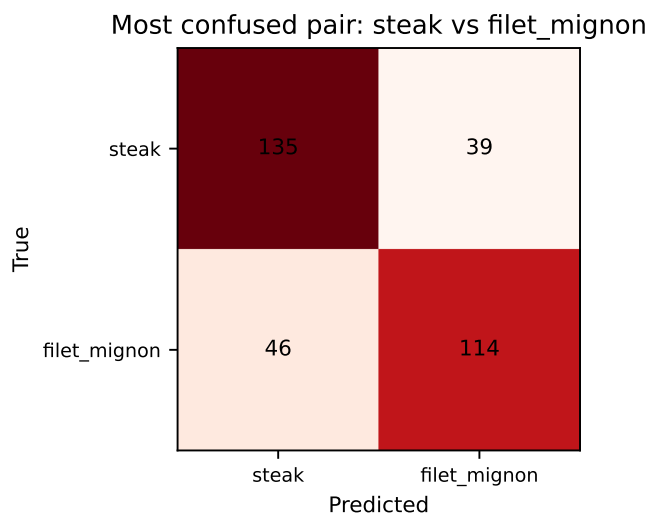


Fig. 5. Visual comparison of highly confused classes (e.g., steak vs. filet mignon) demonstrating the inherent difficulty of fine-grained food classification.